

Networking Fundamentals for Multiplayer Games

Joe Ampfer, ASE 485 Capstone

What I learned with AI while turning Spell Arena into a multiplayer game.

Why Multiplayer Is Hard

In single-player, there is one machine and it is always right.

In multiplayer:

- The game is running on **multiple machines at once**
- The network is **slow** compared to a frame
- When two players disagree about what happened, **someone has to decide**

Most of these slides are really about answering that last question.

Two Architectures

	Dedicated Server	P2P / Host-Client
Cost	Expensive	Free
Cheating	Hard to cheat	Easier to cheat
Latency	Servers can be placed worldwide	Depends on the host's connection
Fairness	Everyone is equal	Host can have an advantage

Same game can run on either. The trade-off is cost vs. trust.

Dedicated Servers

- A machine **only the developer controls** runs the game logic
- **Expensive**, but very useful to prevent cheating
- You can put dedicated servers **all over the world** so players connect to one nearby and minimize latency
- The client only sends inputs and receives state. It never decides what really happened

P2P / Host-Client

- One player's computer **becomes the server**
- Lets you have multiplayer for **free** with no server bill
- Downside: the host can have an **advantage** (zero latency to themselves)
- It's also more **prone to cheating**, because the game logic is running on the host's computer where they can modify it
- Whoever is "the server" is the one who decides what happened. That's the **source of truth**

Remote Procedure Calls (RPCs)

An RPC is basically a way to **call a function that runs on someone else's device**.

- `[ServerRpc]` : the **client** asks the **server** to do something
 - Server validates the request (prevents cheating)
- `[ClientRpc]` : the **server** tells the **clients** to do something
 - e.g. "play this animation," "spawn this VFX"

RPCs are how clients ask for permission, and how the server tells everyone what happened.

State Synchronization

State sync is **duplicating all the important things in your game across every connected device.**

Two examples:

- **Frog example:** if Player 1 spawns a frog, Player 2's computer has to be told to spawn it too, and where
- **Health example:** every computer needs to know the opponent's health so the UI can show it correctly

If state isn't synced, players are literally playing different games.

How State Actually Syncs

Two main mechanisms:

- **RPCs for events** (a spell was cast, a player took damage, a frog spawned)
- **NetworkVariables** (Unity) for **values that need to stay in sync** (health, mana, score)

When a NetworkVariable changes on one machine, **everyone else gets notified**. You can hook listeners onto it so the local game can react: update the UI, play a sound, whatever.

Latency Is Unavoidable

Messages take time to travel. A round trip to the server can easily be **200 to 300ms**.

If you ask the server before doing **anything** the player asked for:

- Player presses a button
- 150ms: request reaches the server
- 150ms: server's response gets back
- Animation finally starts

That's a third of a second of nothing. The game feels **broken**, even though it's working correctly.

Hiding Latency: Action Anticipation

Instead of asking the server "*can I cast this spell?*" and waiting for the answer, you **start the animation first** in anticipation of what the server will say.

- The classic example is **throwing a grenade**. The pin-pull animation plays the moment you click, before the server has confirmed anything
- The animation buys time for the round trip
- The server is **still the authority**. If it says no, you correct
- The client just gets **immediate feedback**

You can't remove latency. You can only hide it.

What I Learned

- **Dedicated vs P2P** is a cost-vs-trust trade-off, not "which is better"
- The **server is the source of truth**. `[ServerRpc]` is how clients ask, `[ClientRpc]` is how the server tells everyone
- **State sync** is just disciplined duplication of important data. Events go through RPCs, values go through NetworkVariables
- **Latency can't be removed, only hidden**. Good games hide it with anticipation instead of stalling the player

Thanks

Questions?